



UNIVERSITY OF PISA

COMPUTER ENGINEERING MASTER DEGREE

Electronics and Communications Systems

Multi-Standard Modulator

Professors:

Marco Moretti

Luca Fanucci

Pietro Nannipieri

Student:

Mykhaylo Semenyshen

Contents

1	Introduction	2
1.1	Project Description	2
1.2	Applications	2
1.3	Background and Alternative Architectures	2
2	System Architecture	3
2.1	High-Level Block Diagram	3
2.2	Pipeline Strategy	4
2.3	Interface and Operating Principle	5
2.4	Error Situations and Design Decisions	5
3	HDL Implementation	6
3.1	Design Hierarchy	6
3.2	Top-Level Entity: MSM	6
3.3	Array Multiplier	8
4	Verification	10
4.1	Test Strategy	10
4.2	Test Plan	11
4.3	Simulation Results	11
5	Synthesis and FPGA Implementation	12
5.1	Setup	12
5.2	Resource Utilization	12
5.3	Timing Analysis	12
5.4	Power Estimation	13
5.5	Warnings Analysis	14
6	Conclusions	14

1 Introduction

1.1 Project Description

The objective of this project is to design a digital circuit implementing a Multi-Standard Modulator (MSM) capable of performing Amplitude Shift Keying (ASK), Frequency Shift Keying (FSK), and Phase Shift Keying (PSK) modulations. The modulator is based on a Numerically-Controlled Oscillator (NCO) that uses a Look-Up Table (LUT) for phase-to-amplitude conversion.

The top-level interface is shown in fig. 1 and detailed in table 1.



Figure 1: Top-level interface of the Multi-Standard Modulator.

Table 1: MSM top-level interface details.

Port	Direction	Width (bits)	Description
clk	Input	1	System clock (125 MHz)
reset	Input	1	Asynchronous active-high reset
FCW	Input	16	Frequency Control Word
PCW	Input	16	Phase Control Word
ACW	Input	4	Amplitude Control Word
y	Output	16	Modulated output signal

The target platform is the Xilinx Zynq XC7Z010-1CLG400C FPGA, available on the Digilent ZyBo development board.

1.2 Applications

NCO-based digital modulators are widely used in software-defined radios, digital up/down converters, radar systems, and signal generators [1]. A single digital circuit can support ASK, FSK, and PSK simply by changing its control inputs, making it well suited for multi-standard platforms.

1.3 Background and Alternative Architectures

The core of the MSM is a Numerically-Controlled Oscillator (NCO). An NCO generates a discrete-time sinusoidal waveform by accumulating a phase increment at each clock cycle and converting the resulting phase into an amplitude [1]. The MSM extends the NCO by adding a phase offset adder (for PSK) and an amplitude multiplier (for ASK).

The phase-to-amplitude conversion inside the NCO can be implemented in different ways:

- **LUT-based:** a ROM stores pre-computed sine values addressed by the phase. A full LUT stores the entire period; a quarter-wave LUT stores only the first quadrant and the rest is reconstructed using the sine's symmetry, reducing memory usage by $4\times$ [1].
- **CORDIC:** an iterative algorithm that computes trigonometric functions using only shifts and additions, avoiding ROM entirely [2].

2 System Architecture

2.1 High-Level Block Diagram

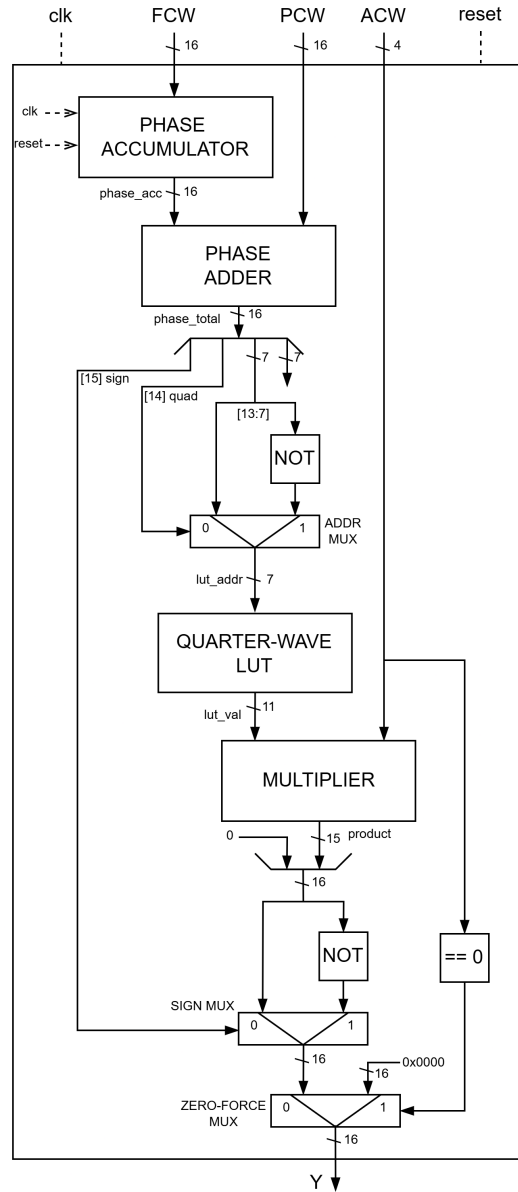


Figure 2: High-level datapath of the Multi-Standard Modulator.

The architecture of the MSM is shown in fig. 2. The datapath consists of the following stages:

1. The **Phase Accumulator** is a 16-bit counter that adds the Frequency Control Word (FCW) to a running phase value at each rising clock edge. It is the only sequential element in the core datapath. Its output is `phase_acc`.
2. The **Phase Adder** is a 16-bit ripple-carry adder that sums `phase_acc` with PCW, producing `phase_total`.
3. The 16-bit `phase_total` signal is then split into three fields:
 - Bit [15] (**sign**): determines the polarity of the output (positive or negative half of the sine). It is routed directly to the Sign MUX.
 - Bit [14] (**quad**): selects the quadrant. It controls the Address MUX.

- Bits [13:7]: the 7-bit base address for the LUT.
 - Bits [6:0]: discarded (below LUT resolution).
4. The **Address MUX** selects between the original 7-bit address and its one's complement (introduces 1 LSB error). This component enables the horizontal mirroring of the first quadrant stored in the LUT. The output is the 7-bit `lut_addr`.
 5. The **Quarter-Wave LUT** is a combinational ROM with 128 entries of 11-bit unsigned sine amplitudes. It stores the first quadrant (0 to $\pi/2$) and maps `lut_addr` to `lut_val`.
 6. The **Multiplier** is an 11×4 unsigned array multiplier that scales `lut_val` by the Amplitude Control Word (ACW), producing the 15-bit unsigned `product`.
 7. The **Sign MUX** selects between `product` and its one's complement (introduces 1 LSB error). This component enables the vertical mirroring of the LUT output.
 8. The **Zero-Force MUX** produces the final 16-bit output `Y` and forces it to `0x0000` whenever `ACW = 0x0`.

2.2 Pipeline Strategy

The purely combinational datapath from the Phase Accumulator output to the final output `Y` is too deep to meet the 125 MHz clock constraint.

Three candidate pipeline insertion points were identified after each major combinatorial component as shown in fig. 3:

- **Position A:** pipeline register A captures `phase_total` signal.
- **Position B:** pipeline register B captures `sign` bit and `lut_val` signals.
- **Position C:** pipeline register C captures `sign` bit and `product` signal.

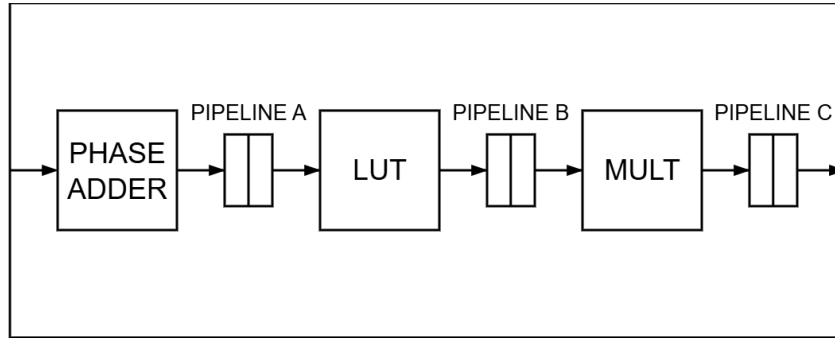


Figure 3: Candidate pipeline insertion points in the MSM datapath.

table 2 summarizes the timing results for different pipeline configurations.

Table 2: Timing results for different pipeline configurations at 125 MHz.

Pipeline	Synth. WNS (ns)	Impl. WNS (ns)	Critical path
None	-5.677	-2.289	Accumulator reg $\rightarrow$ Y output.
A	-1.760	+0.122	Pipe A $\rightarrow$ Y output.
B	-0.310	+0.377	Input $\rightarrow$ pipe B
C	-5.094	-1.661	Accumulator reg $\rightarrow$ pipe C
B + C	+1.150	+0.907	Pipe B $\rightarrow$ pipe C (synth) / Input $\rightarrow$ pipe B (impl)

Pipeline configuration **B** was selected as the final design. It meets implementation timing with a positive slack of +0.377 ns while using only one pipeline register.

2.3 Interface and Operating Principle

The modulated output signal can be expressed as:

$$y[n] \approx \text{ACW} \times \sin\left(\frac{2\pi \cdot \text{FCW} \cdot n}{2^{16}} + \frac{2\pi \cdot \text{PCW}}{2^{16}}\right) \quad (1)$$

where n is the discrete clock cycle index. The three modulation modes are achieved by varying the corresponding control word over time:

- **FSK:** changing FCW modulates the output frequency.
- **PSK:** changing PCW modulates the output phase.
- **ASK:** changing ACW modulates the output amplitude.

Timing behavior. The reset signal is asynchronous and active-high: while asserted, the phase accumulator and pipeline register are cleared and the output becomes zero.

Due to the pipeline structure, different inputs traverse different numbers of register stages before reaching the output, resulting in different latencies as summarized in table 3.

Table 3: Input-to-output latency (in clock cycles) with pipeline configuration B.

Input	Latency (cycles)	Path
FCW	2	Accumulator reg $\rightarrow$ pipeline reg B $\rightarrow$ output
PCW	1	Pipeline reg B $\rightarrow$ output
ACW	0	Combinational to output

All three inputs can be changed simultaneously to produce combined modulation. Since different inputs have different latencies, simultaneous changes may arrive at the output on different clock cycles. In practice this is negligible, as control words are typically held constant for thousands of cycles per symbol and the misalignment is at most 2 cycles.

2.4 Error Situations and Design Decisions

The following edge cases were identified and addressed:

1. **ACW = 0 (zero amplitude):** When ACW = 0, the multiplier output is all zeros. If the sign bit is 1, the one's complement inversion produces 0xFFFF instead of zero. The zero-force mux detects ACW = 0 and forces the output to 0x0000, preventing this artifact.
2. **FCW = 0 (zero frequency):** The phase accumulator stops advancing. The output becomes a DC value determined by the current accumulated phase plus PCW. This is not an error but a valid degenerate case.
3. **High FCW values (near-Nyquist frequencies):** At high FCW values (e.g., 0x8000 produces only 2 samples per sine period), the output waveform degenerates to a few samples per period and no longer resembles a sinusoid due to aliasing. This is mathematically correct and typical of sampled systems so it is up to the user to correctly select FCW values.
4. **Phase accumulator overflow:** The 16-bit accumulator wraps from 0xFFFF to 0x0000 by design, representing the periodic nature of the sine wave.

The following design decisions and their trade-offs are worth noting:

5. **One's complement approximation:** One's complement (bitwise NOT) is used in Address MUX and in the Sign MUX. Both operations introduce an approximation error of up to 1 LSB compared to a true two's complement implementation. This trade-off was accepted because it avoids the need for additional adders, simplifying the hardware.
6. **Unsigned multiplication:** Unsigned multiplication was chosen because it requires simpler hardware than signed multiplication. The sign is handled separately through one's complement inversion after the multiplication.

7. **Output range:** The 16-bit signed output has a maximum positive value of 32,767 (15-bit unsigned magnitude). With an 11-bit LUT (max value 2,047) and a 4-bit ACW (max value 15), the largest product is $2,047 \times 15 = 30,705$, covering 93.7% of the theoretical range. Full coverage is not achievable because 32,767 is not divisible by 15. A non-standard 12-bit LUT with peak value 2,184 would reach 99.98% but requires a wider multiplier for marginal gain.

3 HDL Implementation

3.1 Design Hierarchy

The design follows a structural VHDL approach: each functional block is implemented as a separate entity and instantiated by its parent. The complete component hierarchy is shown in fig. 4.

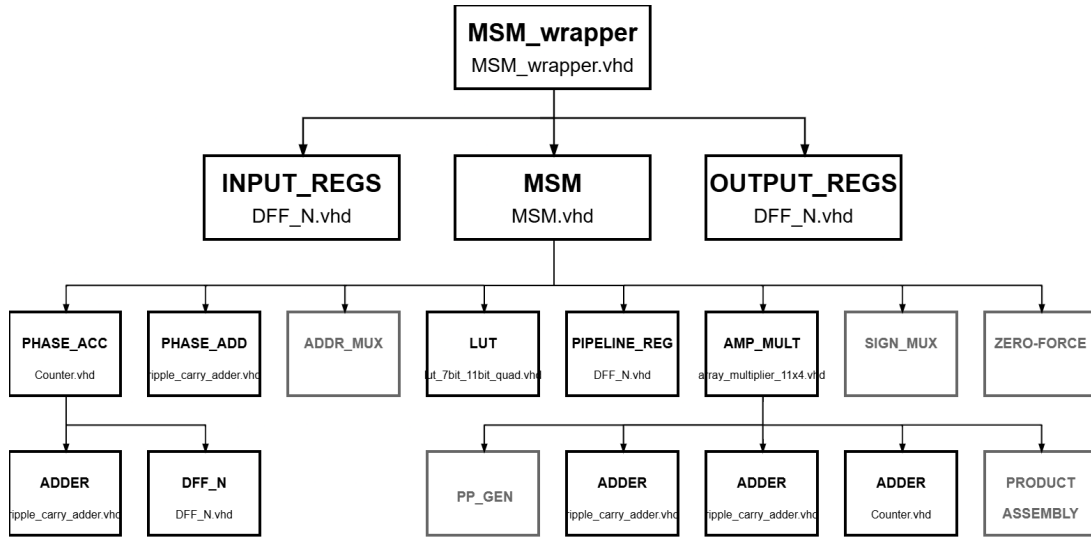


Figure 4: Component hierarchy of the MSM design. Each box shows the instance label and the source file. Gray boxes denote concurrent logic (no separate entity).

The components are organized as follows:

- **MSM_wrapper** is the synthesis top-level. It adds registered I/O boundaries so that Vivado can accurately assess internal timing.
- **MSM** is the modulator core, detailed in section 3.2.
- **array_multiplier_11x4** is a structural multiplier, detailed in section 3.3.
- **Counter**, **DFF_N**, and **ripple_carry_adder** are generic parameterized components reused from course exercises.
- **lut_7bit_11bit_quad** is a 128-entry quarter-wave sine ROM generated by a Python script.

3.2 Top-Level Entity: MSM

The MSM entity interface is shown in listing 1.

```

1 entity MSM is
2   port (
3     clk      : in  std_logic;
4     reset    : in  std_logic;
5     FCW      : in  std_logic_vector(15 downto 0);
6     PCW      : in  std_logic_vector(15 downto 0);
7     ACW      : in  std_logic_vector(3  downto 0);
8     y        : out std_logic_vector(15 downto 0);
9   );
10 end entity;

```

Listing 1: MSM entity interface.

The internal signals are shown in listing 2. Each signal corresponds to a named wire in the block diagram of fig. 2. The only additions are the signals introduced to manage the pipeline register input and output.

```

1  -- Phase accumulator output
2  signal phase_acc_out : std_logic_vector(15 downto 0);
3
4  -- Phase adder output
5  signal phase_total : std_logic_vector(15 downto 0);
6
7  -- LUT address (7 bits from phase_total)
8  signal lut_address : std_logic_vector(6 downto 0);
9
10 -- LUT output (11-bit unsigned sine amplitude)
11 signal lut_output : std_logic_vector(10 downto 0);
12
13 -- Pipeline register input/output: sign_bit & lut_output (12 bits)
14 signal pipe_in : std_logic_vector(11 downto 0);
15 signal pipe_out : std_logic_vector(11 downto 0);
16
17 -- Pipeline register output split
18 signal sign_bit_reg : std_logic;
19 signal lut_output_reg : std_logic_vector(10 downto 0);
20
21 -- Multiplier product (unsigned 15-bit)
22 signal mult_product : std_logic_vector(14 downto 0);
23
24 -- Signed output before zero-force
25 signal y_signed : std_logic_vector(15 downto 0);

```

Listing 2: MSM internal signal declarations.

Phase accumulator and adder. The first two datapath stages instantiate the generic `Counter` and `ripple_carry_adder` components to serve as the `phase accumulator` and `phase adder` (listing 3). The accumulator enable is tied high for continuous operation. The adder carry-out is left open because the 16-bit phase wraps by design.

```

1  -- 1. Phase Accumulator: phase_acc_out += FCW each cycle
2  PHASE_ACCUMULATOR: Counter
3    generic map (N => 16)
4    port map (
5      clk      => clk,
6      a_rst_h  => reset,
7      en       => '1',
8      increment => FCW,
9      cntr_out => phase_acc_out
10   );
11
12 -- 2. Phase Adder: phase_total = phase_acc_out + PCW (carry discarded)
13 PHASE_ADDER: ripple_carry_adder
14   generic map (Nbit => 16)
15   port map (
16     a  => phase_acc_out,
17     b  => PCW,
18     cin => '0',
19     s  => phase_total,
20     cout => open
21   );

```

Listing 3: Phase accumulator and phase adder instantiation.

Address MUX, LUT, and pipeline register. Listing 4 shows the LUT instantiation, its addressing logic and the pipeline register. The address multiplexer is implemented as a conditional signal assignment. The `sign` bit and `lut_output` are grouped in `pipe_in` via concatenation and fed into the pipeline register.


```

1  -- 3. Address MUX: quarter-wave addressing
2  --   quadrant bit [14] = 0 -> forward, 1 -> reverse (one's complement)
3  lut_address <= phase_total(13 downto 7)
4                  when phase_total(14) = '0'
5                  else not(phase_total(13 downto 7));
6
7  -- 4. Quarter-Wave LUT: 128 entries, 11-bit unsigned output
8  LUT_INSTANCE: lut_7bit_11bit_quad
9      port map (
10         address      => lut_address,
11         amplitude_out => lut_output
12     );
13
14  -- 5. Pipeline Register: breaks critical path after LUT
15  --   Packs sign bit [15] together with LUT output to maintain alignment
16  pipe_in <= phase_total(15) & lut_output;
17
18  PIPELINE_REG: DFF_N
19      generic map (N => 12)
20      port map (
21         clk      => clk,
22         a_rst_h  => reset,
23         en       => '1',
24         d        => pipe_in,
25         q        => pipe_out
26     );
27
28  sign_bit_reg <= pipe_out(11);
29  lut_output_reg <= pipe_out(10 downto 0);

```

Listing 4: Address MUX, quarter-wave LUT, and pipeline register B.

Multiplier, sign MUX, and zero-force MUX. The final datapath stages are shown in listing 5. The output of the multiplier instance goes through the sign and zero-force multiplexers to reach the final output of MSM.

```

1  -- 6. Array Multiplier: unsigned 11x4, product = lut_output * ACW
2  AMPLITUDE_MULT: array_multiplier_11x4
3      port map (
4         a      => lut_output_reg,
5         b      => ACW,
6         prod   => mult_product
7     );
8
9  -- 7. Sign MUX: apply polarity using one's complement
10 --   sign=0 -> zero-extend, sign=1 -> NOT(zero-extend)
11 y_signed <= ("0" & mult_product)
12             when sign_bit_reg = '0'
13             else not("0" & mult_product);
14
15 -- 8. Zero-Force: output = 0 when ACW = 0 (prevents -1 artifact)
16 y <= (others => '0')
17     when ACW = "0000"
18     else y_signed;

```

Listing 5: Multiplier instantiation, sign MUX, and zero-force MUX.

3.3 Array Multiplier

The 11×4 unsigned array multiplier computes the final product through partial products and intermediate sums as shown in fig. 5. Its VHDL interface is shown in listing 6.

```

1  entity array_multiplier_11x4 is
2      port (
3         a      : in  std_logic_vector(10 downto 0); -- 11-bit multiplicand
4         b      : in  std_logic_vector(3  downto 0); -- 4-bit multiplier
5         prod   : out std_logic_vector(14 downto 0) -- 15-bit unsigned product
6     );
7  end entity;

```

Listing 6: Array multiplier entity interface.

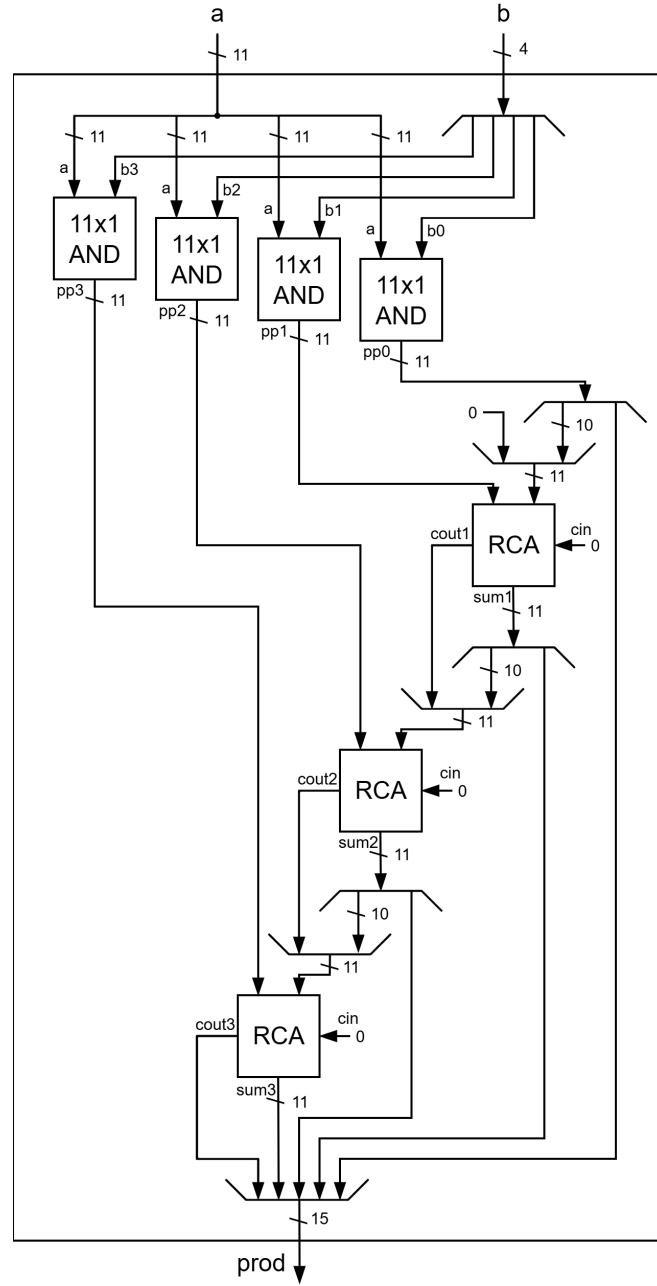


Figure 5: Internal structure of the 11×4 array multiplier.

Partial product generation. Each partial product ppi is an 11-bit vector where bit j equals $a(j)$ AND $b(i)$. The four rows are generated using `for...generate` statements (listing 7).

```

1  -- Partial product generation
2  GEN_PP0: for j in 0 to 10 generate
3    pp0(j) <= a(j) and b(0);
4  end generate;
5
6  GEN_PP1: for j in 0 to 10 generate
7    pp1(j) <= a(j) and b(1);
8  end generate;
9

```

```

10 GEN_PP2: for j in 0 to 10 generate
11     pp2(j) <= a(j) and b(2);
12 end generate;
13
14 GEN_PP3: for j in 0 to 10 generate
15     pp3(j) <= a(j) and b(3);
16 end generate;

```

Listing 7: Partial product generation via generate statements.

RCA chain and product assembly. The first RCA adds `pp1` to the zero-extended, right-shifted `pp0`. Each subsequent RCA adds the next partial product to the carry-out and right-shifted sum of the previous row. At every stage the shifted-out LSB is collected as a product bit. The final product is assembled as `prod = cout3 & sum3(10:0) & sum2(0) & sum1(0) & pp0(0)`, as shown in listing 8.

```

1  -- RCA inputs: shift previous sum right by 1 (LSB goes to product)
2  a_in_row1 <= '0' & pp0(10 downto 1);
3  a_in_row2 <= cout1 & sum1(10 downto 1);
4  a_in_row3 <= cout2 & sum2(10 downto 1);
5
6  -- Row 1: "0" & pp0(10:1) + pp1
7  RCA_ROW_1: ripple_carry_adder
8      generic map (Nbit => 11)
9      port map (
10         a => a_in_row1,
11         b => pp1,
12         cin => '0',
13         s => sum1,
14         cout => cout1
15     );
16
17  -- Row 2: {cout1, sum1(10:1)} + pp2
18  RCA_ROW_2: ripple_carry_adder
19      generic map (Nbit => 11)
20      port map (
21         a => a_in_row2,
22         b => pp2,
23         cin => '0',
24         s => sum2,
25         cout => cout2
26     );
27
28  -- Row 3: {cout2, sum2(10:1)} + pp3
29  RCA_ROW_3: ripple_carry_adder
30      generic map (Nbit => 11)
31      port map (
32         a => a_in_row3,
33         b => pp3,
34         cin => '0',
35         s => sum3,
36         cout => cout3
37     );
38
39  -- Product assembly: collect LSBs from each row + final sum
40  prod(0) <= pp0(0);
41  prod(1) <= sum1(0);
42  prod(2) <= sum2(0);
43  prod(13 downto 3) <= sum3(10 downto 0);
44  prod(14) <= cout3;

```

Listing 8: RCA row chaining and product assembly.

4 Verification

4.1 Test Strategy

The design was verified through behavioral simulation using a stimulus-based testbench (`MSM_tb.vhd`). The verification methodology combines:

- **Visual waveform inspection:** the output waveform y is observed in the simulator and compared against expected behavior (correct sine shape, frequency changes, phase shifts, amplitude scaling and reset behavior).
- **Automatic assertions:** two concurrent assertion processes run during simulation:
 - *Zero-force check:* verifies that $y = 0$ whenever $ACW = 0$.
 - *Metavalue check:* verifies that y never contains uninitialized values (X or U).

The test plan covers normal operation for all three modulation modes, reset behavior, and edge cases as detailed in table 4.

4.2 Test Plan

Table 4: Test plan summary.

#	Category	Description	Expected
1	Reset	Reset with non-zero inputs	$y = 0$
2	FSK	Basic sine after reset	Basic sine
3	Edge case	FCW=0, PCW=90° (DC)	Constant output
4	FSK	Low frequency	Base-rate sine
5	FSK	Double frequency	Double-rate sine
6	PSK	PCW=180°	+90° offset
7	PSK	PCW=90°	−90° offset
8	PSK	PCW=270°	+180° offset
9	ASK	ACW=0 (zero-force)	$y = 0$
10	ASK	ACW=1 (minimum)	Smallest sine
11	ASK	ACW=8 (mid)	Half amplitude
12	ASK	ACW=15 (max)	Full amplitude
13	Edge case	Change FCW, PCW, ACW together	Clean transition
14	Reset	Reset mid-operation	$y \rightarrow 0$
15	Reset	Resume after reset	Normal sine
16	Edge case	High frequency	Aliased waveform
17	Edge case	Slow sine, accumulator wrap	No discontinuity

4.3 Simulation Results

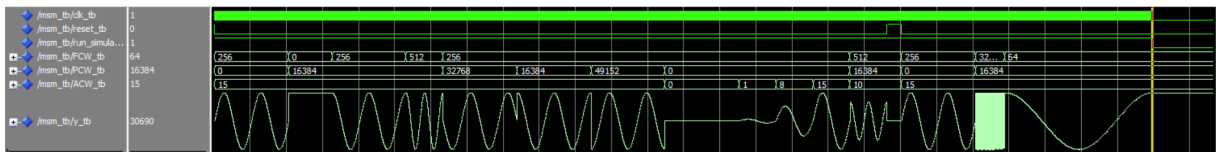


Figure 6: Simulation waveform showing all 17 test cases.

fig. 6 shows the complete simulation output. The waveform begins flat at zero during reset (Test 1), then a basic sine appears once reset is released (Test 2). Setting FCW to zero freezes the phase accumulator, producing a constant DC level (Test 3). Tests 4–5 demonstrate FSK: the sine runs at a base rate, then its period visibly halves when FCW is doubled. In Tests 6–8, different values of PCW shift the waveform by 90°, −90°, and 180° respectively. The ASK tests (9–12) sweep ACW from zero to maximum. Test 13 changes all three control words at once, producing a sudden transition. Reset is then asserted mid-operation (Test 14), forcing the output back to zero, and released again (Test 15) to confirm normal recovery. Finally, a high frequency produces the expected and degenerate aliased waveform of only 2 samples per period (Test 16), and a slow sine completes a full accumulator wrap with no discontinuity (Test 17). Both automatic assertions completed without errors throughout the simulation.

5 Synthesis and FPGA Implementation

5.1 Setup

Synthesis and implementation were performed using Xilinx Vivado targeting the Zynq XC7Z010-1CLG400C device on the ZyBo development board. A clock constraint of 125 MHz (period = 8 ns) was applied to the `clk` port. The design was synthesized and implemented in out-of-context mode.

5.2 Resource Utilization

Utilization

Post-Synthesis

Post-Implementation

Graph

Table

Resource	Utilization	Available	Utilization...
LUT	111	17600	0.63
FF	80	35200	0.23

Figure 7: Post-implementation resource utilization.

Figure 7 shows the post-implementation resource utilization. The design uses 111 LUTs out of 17 600 available (0.63%) and 80 flip-flops out of 35 200 (0.23%). This confirms that the modulator is a very lightweight design, occupying less than 1% of the FPGA fabric and leaving ample room for additional logic or integration into a larger system.

5.3 Timing Analysis

Timing	Setup
Worst Negative Slack (WNS):	0.377 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	44
Implemented Timing Report	

Figure 8: Post-implementation timing summary.

The post-implementation timing report (Figure 8) shows a Worst Negative Slack (WNS) of +0.377 ns with zero failing endpoints, confirming that all timing constraints are met at 125 MHz. The theoretical maximum achievable clock frequency is:

$$f_{\max} = \frac{1}{T_{\text{clk}} - \text{WNS}} = \frac{1}{8.0 - 0.377 \text{ ns}} = \frac{1}{7.623 \text{ ns}} \approx 131.2 \text{ MHz} \quad (2)$$

Name	Slack	Levels	High Fanout	From	To
↳ Path 1	0.377	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[1]/D
↳ Path 2	0.400	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[0]/D
↳ Path 3	0.410	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[8]/D
↳ Path 4	0.439	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[5]/D
↳ Path 5	0.463	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[9]/D
↳ Path 6	0.484	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[4]/D
↳ Path 7	0.496	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[3]/D
↳ Path 8	0.537	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[2]/D
↳ Path 9	0.553	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[6]/D
↳ Path 10	0.574	9	21	PCW_INPUT_REG/q_reg[1]/C	MSM_CORE/PIPELINE_REG/q_reg[10]/D

Figure 9: Ten worst-case setup paths after implementation.

As shown in Figure 9, all ten worst-case paths originate from the PCW input register (PCW_INPUT_REG) and terminate at the pipeline register (MSM_CORE/PIPELINE_REG). These paths traverse the phase adder, the quarter-wave LUT, and arrive at the pipeline stage, representing the longest combinational chain in the design. Longer than the path that originates at the pipeline register, goes through the multiplier, sign mux and zero-force mux to reach the output register.

5.4 Power Estimation

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: **0.097 W**
Design Power Budget: **Not Specified**
Power Budget Margin: **N/A**
Junction Temperature: **26.1°C**
Thermal Margin: 58.9°C (5.0 W)
Effective θ_{JA} : 11.5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: **Medium**
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

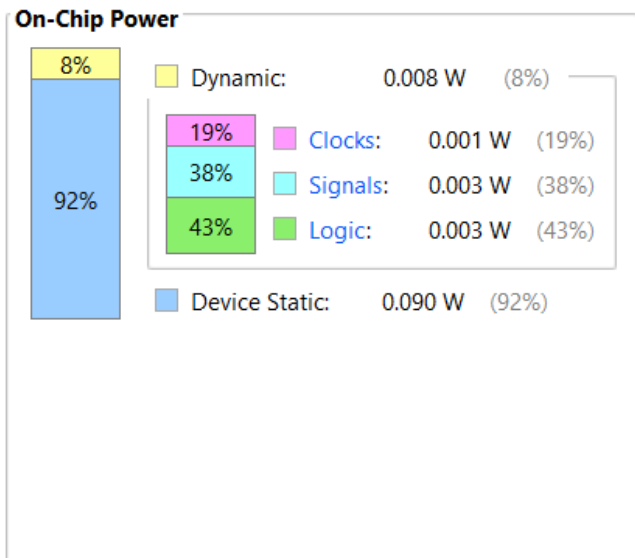


Figure 10: Post-implementation power estimation.

The Vivado power analysis (Figure 10) reports a total on-chip power consumption of 0.097 W. Device static power accounts for 0.090 W (92% of the total), which represents the leakage current inherent to the FPGA and is independent of the design. Dynamic power is only 0.008 W (8%), subdivided into clock distribution (19%), signal switching on routing wires (38%), and logic switching in LUTs and flip-flops (43%). The estimated junction temperature is 26.1 °C with a thermal margin of 58.9 °C, indicating no thermal concerns.

5.5 Warnings Analysis

Synthesis completed with no warnings. Implementation produced 41 warnings, all of which are expected consequences of the out-of-context mode and do not affect functional correctness or timing validity. They can be grouped into the following categories:

DRC – PS7 cell not instantiated. The Zynq device includes a hard-wired Processing System (PS7) block. Vivado warns that it is not used in the design, which prevents default device configuration. This warning is irrelevant in out-of-context mode, where no bitstream is generated and the PS7 is not needed.

Timing 38-242 – HD.CLK_SRC not set on clk. In out-of-context mode, Vivado does not know which physical clock buffer drives the `clk` port, so it cannot estimate the absolute clock insertion delay from the source. The relative skew between internal flip-flops is still computed from the actual clock routing, so the internal setup/hold analysis remains valid. This warning would only become relevant when integrating the module into a top-level design.

Route 35-198 – Ports missing HD.PARTPIN_LOCS. In out-of-context mode, the top-level ports (FCW, PCW, ACW) are not assigned to physical FPGA pins, so Vivado cannot estimate the signal delay from the pins to the module. This has no effect on internal timing, which is fully routed and remains accurate.

6 Conclusions

This report presented the design, implementation, and verification of a Multi-Standard Modulator supporting ASK, FSK, and PSK modulations on a Zynq XC7Z010 FPGA. The architecture centers on a LUT-based Numerically-Controlled Oscillator extended with a phase offset adder and an amplitude multiplier, enabling all three modulation modes through a single, unified datapath.

A key design challenge was meeting the 125 MHz timing constraint. A systematic exploration of pipeline insertion points showed that a single register after the quarter-wave LUT (configuration B) was sufficient, achieving a post-implementation slack of +0.377 ns. The final design occupies only 111 LUTs and 80 flip-flops — less than 1% of the available fabric — confirming that the modulator is lightweight enough for integration into larger systems.

Behavioral simulation across 17 test cases verified correct operation for all modulation modes, reset behavior, and edge cases including the zero-amplitude artifact and phase accumulator wraparound. Both automatic assertions completed without errors.

Several trade-offs were accepted in favor of hardware simplicity, notably the use of one's complement approximations in the address and sign multiplexers, each introducing up to 1 LSB of error. Future improvements could include switching to two's complement for exact symmetry, using a larger LUT for better output quality, and making the bit widths generic for reuse in other designs.

References

- [1] Analog Devices, “A technical tutorial on digital signal synthesis,” tech. rep., 1999. Available at https://www.analog.com/media/en/training-seminars/tutorials/450968421DDS_Tutorial_rev12-2-99.pdf.
- [2] J. E. Volder, “The CORDIC trigonometric computing technique,” *IRE Transactions on Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, 1959.